

Technology Decision Reference — Staff+

One page per technology. The mechanism that drives the tradeoff, when to reach for it, when it flips, and the line to say in the room.

This sheet answers "which technology and why" — the axis a pattern-organized system design sheet doesn't cover. Every entry follows the same skeleton so it's scannable under pressure: **mechanism** → **reach for / avoid** → **numbers** → **CAP** → **interview line** → **pushback**. The interview line is the choosing-not-pattern-matching signal; the pushback is where the obvious answer reverses, which is the staff tell.

Verified August 2026 against primary sources (AWS, Apache, PostgreSQL docs). Facts that changed recently — and that older study material still gets wrong — are marked ● **changed** in place.

On the numbers: these are interview anchors — order-of-magnitude figures to reason with out loud, not spec-sheet facts. Treat ranges as talking points, not exact values to be tested against.

CONTENTS

01 PostgreSQL — the default; ACID, joins, one node does more than you think

02 Cassandra — query-first, masterless, multi-region writes; you give up joins

03 DynamoDB — managed Dynamo-lineage KV; write-scaling with no cluster to run

04 Redis — in-memory speed; cache, counter, lock, queue, ephemeral state

05 Kafka — durable ordered log; decouple, buffer, replay, fan-out

06 Flink — stateful stream processing; windows, joins, exactly-once state

07 Message Queue (SQS / RabbitMQ) — per-message work dispatch; ack, retry, DLQ

08 S3 / Blob Storage — infinite cheap bytes; the bytes never touch your API

09 CDN — push bytes to the edge; the answer to "global" and "read-heavy static"

10 Vector DB — ANN over embeddings; the retrieval half of RAG

11 Workflow Orchestrator — durable fan-out/fan-in, exactly-once continuation

12 ZooKeeper / etcd — consensus-backed coordination; leader election, locks, config

13 Elasticsearch — inverted index; full-text + faceted search, not a primary store

14 OLAP / Columnar — scan-and-aggregate read model for analytics at scale

15 Decision matrix — workload → pick, at a glance

01 PostgreSQL

The correct default. Reach for something else only when you can name the specific thing Postgres can't do at your scale.

MECHANISM

B-tree indexes, MVCC, single-leader replication. Crucially, Postgres does not update in place — an `UPDATE` writes a new row version and marks the old one dead, so readers get a consistent snapshot without blocking writers. `VACUUM` reclaims the dead versions (and is why bloat, HOT updates, and index write-amplification are things you tune). One primary takes writes; read replicas stream the WAL (write-ahead log), asynchronously by default. Write ceiling is set by local WAL fsync throughput and hot-row lock contention, not by replicas.

● **Version note.** PG 18 (Sept 2025) added an asynchronous I/O subsystem (faster sequential scans, bitmap heap scans, vacuum), B-tree skip scan, and native `uuidv7()` for time-ordered keys with better index locality. The process-per-connection model is unchanged, so the pooling advice below still holds.

REACH FOR IT WHEN

- + You need multi-row / multi-table ACID transactions — money, inventory, multi-entity publish.
- + Read patterns are ad-hoc or unpredictable — the planner + indexes handle new `WHERE` clauses for free.
- + You need joins, aggregations, secondary indexes.
- + Scale is anything below "genuinely global + huge." Which is almost always.

AVOID / AUGMENT WHEN

- Write volume genuinely exceeds one primary (see the trigger numbers) → shard or LSM store.
- You need active-active multi-region writes — retrofitted, painful.
- Append-only firehose (metrics, events) — LSM fits better.
- **Connection storms** — each connection is ~a backend process (megabytes of RAM); a few thousand direct connections will exhaust it. Put PgBouncer in front. The single most common real-world Postgres scaling gotcha, and it hits well before you run out of raw throughput.

TRIGGERS TO BREAK OUT (NAME THE NUMBER)

The trigger is almost always a specific hot table or row, or a write pattern — not aggregate database size. 100M+ rows on one primary is routine.

- > **< ~5k writes/s to one logical table** → you're fine on one primary. Don't move; you're solving a problem you don't have.
- > **~5k–15k writes/s sustained** → tune first: batch inserts, offload the hot counter to Redis, faster-fsync disk (NVMe), fewer indexes on the hot table (HOT updates). Most "Postgres is slow" cases die here without an architecture change.
- > **~15k–50k writes/s sustained to one table** → shard Postgres by a good key (app-level or Citus). You keep SQL and transactions within a shard; you give up cross-shard joins and painless rebalancing.
- > **> ~50k writes/s sustained to one table**, append-only firehose (metrics/events), or active-active multi-region writes → move that hot table to an LSM / wide-column store (Cassandra / DynamoDB)

or a purpose-built TSDB. Here the write *pattern*, not the size, forces the change.

- > **A single hot row taking > ~500–1k serialized updates/s** (a global counter, one inventory row) → the ceiling is row-lock contention, not table throughput. Offload that one value to Redis (`INCR`) or shard the counter; the rest of the table stays in Postgres.

DURABILITY LAYERS (KNOW COLD)

- > **Local WAL fsync** survives a crash (power loss, panic) — recovery replays it. It does *not* survive losing the disk/node.
- > **Synchronous replication** (`ANY 1` of several standbys, ideally another AZ) means no acknowledged write lives on a single disk. Use `ANY 1` quorum-with-slack, not a single named standby — requiring a specific replica means its failure blocks all writes.
- > **WAL archiving to object storage + PITR** survives logical corruption (a bad `DELETE` replicates faithfully to every replica; only point-in-time recovery saves you).

NUMBERS TO ANCHOR

Point read (indexed)	< 1 ms
Writes/s, one primary	~5k–15k
Shard trigger (one table)	~15k–50k w/s
LSM / firehose trigger	> ~50k w/s
Serialized updates / hot row	~500–1k/s
Rows before sharding hurts	~10–100M+
Connections	pool it (~100s)

CAP / CONSISTENCY

CP-leaning, but really ACID on one node + tunable replication. Transactional consistency (isolation up to `SERIALIZABLE`) is a native capability. Replication consistency is a separate knob: `async` replicas are eventually consistent; `synchronous_commit = remote_apply` or reading the primary gives strong.

INTERVIEW LINE

I default to Postgres and make something else justify itself — transactions, joins, and query flexibility for free, one primary does 5–15k writes/s, reads scale on replicas. I only move off it when I can name the exact property it can't provide at my scale — a specific hot table past ~15–50k writes/s, a firehose, or a multi-region write — and I pool connections and tune the hot table before I touch the architecture.

PUSHBACK / WHEN IT FLIPS

"Postgres doesn't scale" is usually wrong — a single hot row, a multi-region write, or connection exhaustion flips it, not raw size. Sharded Postgres by a good key gets most of Cassandra's write distribution; what you lose is automatic rebalancing and painless geo-replication, not the sharding itself.

02 Cassandra

Query-first, masterless, born multi-region. You trade joins and ad-hoc queries for cheap global replication and painless rebalancing.

MECHANISM

LSM tree + consistent hashing + masterless replication. Writes append to a memtable + commitlog, flush to immutable SSTables — sequential I/O, so the write ceiling is high. Compaction merges SSTables in the background. Consistent hashing (with vnodes) places partitions on a ring — adding a node auto-rebalances token ranges. Any node can coordinate a write and forward to the replica set; there's no special primary. Consistency is tunable per query (`ONE` , `QUORUM` , `ALL`); multi-datacenter replication is a config declaration.

REACH FOR IT WHEN

- + Access is known, key-based lookups — "everything for this id," "this partition sorted by time."
- + Global reads needing local-region latency — multi-DC replication out of the box.
- + Very high write throughput, append-heavy.
- + You want capacity-by-adding-nodes with no manual reshard.

THE MODEL TAX (KNOW COLD)

- No joins, and no genuinely ad-hoc queries. Cassandra 5.0 (Sept 2024) added **Storage Attached Indexes (SAI)** — real secondary indexes on most column types — so "query by non-partition column = full scan" is now too strong. But a query that doesn't hit the partition key still fans out across the ring: cheaper than before, never free.
- One table per query — denormalize, dual-write, own consistency across copies yourself.
- No general transactions. LWT (Paxos) does single-partition compare-and-set only, and it's slow. General-purpose transactions (Accord, CEP-15) have been in development for years and are *still not GA* — don't claim them in a design.
- Incomplete clustering key silently upserts (clobbers rows).
- **Tombstones:** deletes don't remove data, they write a marker that lingers until compaction. Read across many tombstones and latency craters — this is why Cassandra is a terrible queue (write-then-delete churns tombstones). The #1 footgun after clustering-key upserts.
- **Hot partition:** throughput and storage are spread by partition key, so one oversized or over-read partition (a celebrity user, a global bucket) pins load to one replica set while the cluster looks idle. Keep partitions bounded — a working ceiling is ~100 MB / ~100k rows — and bucket by time or hash suffix when a natural key grows unbounded.

● MUTATION COST – THE READ TAX

- **Writes are cheap, and an update is just a write** — appended to the memtable, no read-before-write. Cassandra absorbs update volume fine; the write path is not where you pay.
- **The cost lands on reads.** SSTables are immutable, so each update leaves another fragment of that row in another SSTable. A read merges fragments across SSTables — bloom filters skip most, not all — so an overwrite-heavy row gets steadily more expensive to read until compaction consolidates it.
- **Deletes are the worst case:** a tombstone is a write that makes reads slower and only disappears after compaction plus `gc_grace`.
- **So the fit isn't "written rarely" — it's mutated rarely.** Write-once-and-read and append-and-read are ideal; repeated read-modify-write on one row is where the tax shows up, as read latency and compaction I/O.

COMPACTION STRATEGY IS THE LEVER

- **STCS** (historical default) — merges similarly-sized SSTables. Cheap writes (~2–4× write amplification), high read and space amplification. Suits write-heavy / append-only.
- **LCS** — non-overlapping levels, roughly one SSTable per level per read. The answer for update-heavy or read-heavy tables, at ~10–30× write amplification.
- **TWCS** — time-windowed, for TTL'd time-series; expires whole windows without dragging reads through tombstones.
- **UCS** (5.0) — unified; configurable to behave like any of the above and changeable in flight. The current recommended default.
- Naming the compaction strategy when you propose Cassandra is a strong signal — it's where the update-vs-read tradeoff actually gets decided.

NUMBERS TO ANCHOR

Write throughput	very high, scales out
Single-partition read	low ms
Add capacity	join a node
Consistency	tunable per query
Strong reads	$R + W > N$
Partition size target	< ~100 MB
Write amp: STCS / LCS	~2–4× / ~10–30×
Default compaction (5.0)	UCS

CAP / CONSISTENCY

AP by design, tunable toward CP. `QUORUM` reads + `QUORUM` writes satisfy $R + W > N$, so the read overlaps the latest write → strong consistency, at the cost of latency and availability during partitions. Even at `ALL`, you get replication consistency, never transactional — no "update these three rows or none."

INTERVIEW LINE

Cassandra fits keyed lookups that are read globally and *mutated* rarely — append-and-read, not read-modify-write. Its edge over sharded Postgres isn't write throughput at my volume — it's that consistent hashing makes rebalancing automatic and multi-region replication a config line, not a project. I accept no joins and eventual consistency, tunable to strong via `QUORUM/QUORUM`. If the table does take frequent updates I'd move it to leveled compaction and say why: immutable SSTables mean every update fragments the row, and reads pay to merge those fragments.

PUSHBACK / WHEN IT FLIPS

Below genuine global scale, Postgres is the better default even for keyed data — you keep transactions and query flexibility and defer the operational tax (repair, compaction, tombstone tuning, eventual-consistency reasoning). Cassandra is right when scale × geo-distribution × access-pattern-simplicity all hold at once. And never model a queue or high-churn delete workload on it — tombstones will punish you. Be precise about the write story too: Cassandra absorbs writes beautifully, updates included. The penalty for an overwrite-heavy table lands on read latency and compaction I/O, not on write throughput — "Cassandra is bad at these writes" is the wrong diagnosis and an interviewer will catch it.

03 DynamoDB

The managed, serverless member of the Dynamo family. You buy Cassandra-style write-scaling and multi-region replication without running a cluster — and pay in partition-key discipline and per-request pricing.

MECHANISM

The partition key is hashed to place an item on a partition; an optional sort key orders items within a partition. Storage auto-shards as data and throughput grow — no nodes to manage. Reads default to eventually consistent (cheaper, served from any replica); you opt into strongly consistent reads per request (more cost, leader replica; cross-Region only under MRSC — see CAP). Capacity is on-demand (pay-per-request, auto-scales to spikes) or provisioned (RCU/WCU you set, cheaper at steady load). Secondary indexes: **GSI** (different partition key, its own throughput, eventually consistent) and **LSI** (same partition key, alternate sort key, fixed at table creation). DynamoDB Streams emits an ordered per-item change log — CDC for triggers, replication, and fan-out.

REACH FOR IT WHEN

- + Known key-based access at scale and you want zero ops — no cluster to run, patch, or rebalance.
- + Serverless / spiky workloads — on-demand capacity tracks traffic without a capacity plan.
- + Predictable single-digit-ms latency at any scale for keyed lookups.
- + Multi-region active-active (global tables) as a config option, not a project.

THE MODEL TAX (KNOW COLD)

- **Hot partition:** throughput is spread across partitions by key. A skewed key (one celebrity, one tenant) concentrates load on a single partition and throttles — even when the table has spare total capacity. Same failure class as Cassandra's hot partition; pick a high-cardinality, evenly-accessed partition key.
- **Item size cap 400 KB** — large blobs go to S3 with the key stored in Dynamo.
- **Single-table design** — model access patterns up front and overload one table with composite keys; ad-hoc queries aren't a thing.
- **No joins; transactions are limited** — `TransactWriteItems` caps at 100 actions and 4 MB aggregate, costs ~2× throughput, works only within one account and Region, and is *unavailable entirely* on MRSC global tables. Not a substitute for relational transactions.
- **Cost surprises** — `Scan` and unbounded GSIs get expensive fast. Model your reads as key lookups; never design around scanning.

NUMBERS TO ANCHOR

Latency (keyed)	single-digit ms
Item size cap	400 KB
Read default	eventual (strong opt-in)
Transaction	≤ 100 actions / 4 MB
Global tables	MREC or MRSC
MRSC topology	exactly 3 Regions
Scale	auto-shard, no ops

CAP / CONSISTENCY

AP by default, CP on request. Within a Region, reads are eventually consistent unless you set `ConsistentRead`. Across Regions there are now two modes. **MREC** (multi-Region eventual consistency) is the long-standing default: last-writer-wins, async replication. **MRSC** (multi-Region strong consistency) went GA on 30 June 2025 and acknowledges a write only once it is durable across the Region set — RPO zero, and any Region can read the latest value.

MRSC is tightly constrained, and the constraints are the interesting part: exactly three Regions (three replicas, or two replicas plus a DynamoDB-managed **witness** that holds data but serves no traffic); all within one Region set (US / EU / AP — no mixing); the table must be empty when you enable it; the transaction APIs are unavailable; and writes and strongly-consistent reads pay a cross-Region round trip. Conflicting concurrent writes surface as `ReplicatedWriteConflictException`.

INTERVIEW LINE

DynamoDB is the managed Dynamo-lineage store: partition-key scaling and global replication with no cluster to operate, in exchange for designing every access pattern into the key up front. Reads are eventually consistent unless I ask for strong per request, and my one real job is a high-cardinality partition key so I don't hot-spot one partition while the table looks under-utilized. If the requirement is a global write with zero RPO, I'd name MRSC global tables and its cost in the same breath: three Regions inside one Region set, cross-Region latency on every write, and no transaction APIs.

PUSHBACK / WHEN IT FLIPS

Below real scale, Postgres still wins — you keep joins, transactions, and the ad-hoc queries Dynamo's single-table model forbids. DynamoDB vs Cassandra is mostly buy vs run: same data model and tradeoffs, DynamoDB is managed/ serverless with per-request cost, Cassandra is self-hosted with fixed cluster cost. The classic incident is the hot partition — a skewed key throttles one partition while the table looks under-provisioned. And retire the stale claim that global tables are eventually consistent full stop; MRSC has been GA since mid-2025. The real question is whether the workload will pay a cross-Region round trip on every write to get it — usually only money and inventory will.

04 Redis

In-memory, microsecond ops. A Swiss-army knife: cache, counter, lock, rate limiter, ephemeral queue, leaderboard. Rarely your source of truth.

MECHANISM

RAM-resident data structures with single-threaded command execution — every command (and every Lua script) runs atomically with no locks, which is the load-bearing property. (Modern Redis uses I/O threads for network, but execution of the actual commands is still serialized, so the atomicity guarantee holds.) Durability is optional: RDB snapshots + AOF append log (everysec is the usual). Replicas + Sentinel/Cluster for HA. Treat it as a fast cache that can persist, not a database that happens to be fast.

REACH FOR IT WHEN

- + Cache in front of a slower store (the default use).
- + Atomic counters — INCR / DECR for rate limits, inventory admission, fan-in completion.
- + Distributed lock (short-lived; SET NX PX + fencing token).
- + ZSET for leaderboards, waiting-room ordering, timer queues.
- + Ephemeral state with TTL auto-expiry (sessions, holds).

CACHE PATTERNS TO NAME

- > **Cache-aside (lazy):** app checks cache, on miss reads DB and populates. The default. Risk: stale entries → set TTLs / invalidate on write.
- > **Write-through / write-behind:** write hits cache (and DB sync/async). Stronger freshness, more coupling.
- > **Stampede / thundering herd:** a hot key expires and thousands of requests hit the DB simultaneously to refill. Mitigate with a short lock around the refill (one request repopulates), request coalescing, or probabilistic early expiry. Directly relevant to any "hot cached value" design.

AVOID / CAREFUL WHEN

- It's your only copy of important data — a failover to a lagging replica loses recent writes.
- Dataset > RAM. It's memory-bound; that's the cost model.
- **TTL as a business trigger:** expiry deletes a key, it doesn't run your compensation. Keyspace notifications are at-most-once — don't rely on them for correctness.

● REDIS VS VALKEY (KNOW THE ONE-LINER)

- > Redis left BSD in March 2024 for RSALv2/SSPL; the Linux Foundation forked 7.2.4 as **Valkey** (BSD, backed by AWS/Google/Oracle); Redis 8 (May 2025) added AGPLv3 as a third option.
- > AWS ElastiCache now defaults new clusters to Valkey and prices it below Redis. Valkey is wire-compatible — same commands, same clients.
- > Nothing on this page changes between them. Just don't be surprised when an interviewer says "Valkey."

NUMBERS TO ANCHOR

Op latency	~0.1–0.5 ms
Throughput, one node	~100k+ ops/s
Bound	RAM + network

CAP / CONSISTENCY

CP within a shard for single-key ops (atomic execution), but the durability/HA story is weak by default — async replication + Sentinel failover can lose recently-acknowledged writes. Redis Cluster shards by key; multi-key atomicity only within a hash slot.

INTERVIEW LINE

I reach for Redis when I need one thing very fast and can tolerate losing it: a cache, an atomic counter, a TTL'd hold. Serialized command execution means every op and every Lua script is atomic, which is why it's the right place for admission control and rate limiting. I keep the durable truth in Postgres behind it, and I plan for cache stampede on hot keys.

PUSHBACK / WHEN IT FLIPS

For a hold/expiry pattern, Redis TTL only wins if the key *is* the hold (its disappearance is the release). If Postgres owns the counter, a DB sweeper beats Redis notifications — durable and self-healing. Use Redis for the ticket-onsale scale (100k holds/s, visible countdown); keep the sweeper for the hotel scale.

05 Kafka

A durable, ordered, replayable log. Not a queue you drain — a commit log consumers read at their own offset. The backbone for decoupling and buffering.

MECHANISM

Partitioned append-only log; consumers track offsets. Each topic splits into partitions; order is guaranteed within a partition, not across. Messages persist for a retention window — consumers replay by rewinding their offset. Producers key messages to control partition (same key → same partition → ordered). Delivery is at-least-once by default; consumers must be idempotent.

REACH FOR IT WHEN

- + Decouple producers from consumers — upload service emits, transcode fleet consumes independently.
- + Buffer a firehose — absorb spikes, consumers process at their own rate (load leveling).
- + Fan-out one event to many consumer groups (analytics + search index + notifications).
- + Event sourcing / CDC — the log is the source of truth; replay to rebuild.

OPERATIONAL REALITIES (KNOW COLD)

- › **Partition count is your parallelism ceiling:** within a consumer group, consumers $\leq$ partitions (extra consumers sit idle). You pick partition count up front and it's awkward to reduce — size for peak parallelism.
- › **Consumer-group rebalancing:** when a consumer joins or leaves, partitions are reassigned. Under the classic protocol this is a stop-the-world pause, and frequent rebalances (flapping consumers, long processing) are a classic Kafka incident. The new protocol (KIP-848, GA in Kafka 4.0) makes reassignment incremental and broker-driven, so the global pause is largely gone — name which protocol you're assuming.
- › **KRaft, not ZooKeeper:** Kafka 4.0 (March 2025) removed ZooKeeper outright. Metadata lives in an internal Raft quorum of controller nodes. Describing Kafka as needing a ZooKeeper ensemble now describes a version nobody deploys new.
- › **Ordering vs. parallelism tension:** you get order within a partition, so more partitions = more parallelism but coarser ordering. Key by the entity whose order matters.

AVOID / CAREFUL WHEN

- You need priority or delayed/scheduled delivery — still absent; that's a task queue. (Per-message ack is no longer a clean separator — see share groups.)
- You need global ordering — you only get per-partition.
- Low-volume simple job dispatch — Kafka is operational weight you may not need.

● SHARE GROUPS (KIP-932) – KAFKA LEARNED TO QUEUE

- › Early access in 4.0, preview in 4.1, GA alongside 4.2: a **share group** lets multiple consumers read the *same* partition cooperatively, with per-message acknowledgement, a broker-side delivery count, and poison messages archived after a configurable limit (default 5).
- › Consumer count is no longer capped by partition count for those groups — the parallelism ceiling above applies to classic consumer groups.
- › It narrows the log-vs-queue gap but doesn't close it: no priority, no delay, and it's new enough that "use a real queue" is still the safe interview answer. Knowing it exists is the differentiator.

NUMBERS TO ANCHOR

Throughput	very high (MB/s+/part.)
Ordering	per-partition only
Delivery	$\geq$ once (be idempotent)
Parallelism	$\leq$ partitions (classic)
Metadata (4.0+)	KRaft, no ZooKeeper
Retention	hours → forever

CAP / CONSISTENCY

CP-leaning: a partition has a leader + ISR (in-sync replicas); `acks=all` waits for the ISR, trading latency for no-loss. Exactly-once semantics (EOS) exist but only *within* Kafka — read-process-write across Kafka topics. It does not extend exactly-once to external side effects (your DB, an API), which still need idempotency.

INTERVIEW LINE

I use Kafka to decouple and buffer: producers append, consumer groups read independently at their own offset, and retention lets me replay to rebuild a downstream or add a consumer. I key by entity for per-partition ordering, size partitions for peak consumer parallelism, and make consumers idempotent because delivery is at-least-once — Kafka's exactly-once doesn't cover my database write.

PUSHBACK / WHEN IT FLIPS

Reaching for Kafka on a simple job queue is over-engineering — if you want per-message ack, retry, priority, and a DLQ without replay or fan-out, a task queue is simpler. Kafka earns its operational cost when you need replay, multiple independent consumers, or firehose buffering — name which one. Note the ground has shifted slightly: share groups give Kafka per-message ack, so the honest distinction is now replay and fan-out (log) versus priority, delay, and mature DLQ tooling (queue). And Kafka only transports the stream; the moment you need to window, aggregate, or join it, that's Flink's job, not a hand-rolled consumer.

06 Flink

Stateful stream processing with event-time windows and exactly-once state. Kafka moves the events; Flink computes over them — running aggregations, joins, and windows on the stream itself.

MECHANISM

A dataflow graph of operators runs continuously over unbounded streams. Operators hold **keyed state** (per-key aggregates, join buffers) in a local state backend — RocksDB on local disk when state is large. Fault tolerance is **distributed checkpointing**: Chandy-Lamport barriers flow through the graph and snapshot each operator's state consistently; on failure the job rewinds to the last checkpoint and replays from the source offset → exactly-once state. **Event-time** processing with **watermarks** handles out-of-order and late events: a watermark asserts "no more events older than T," which fires time windows correctly regardless of arrival order — versus processing-time (wall clock, simpler, but wrong under lateness).

REACH FOR IT WHEN

- + Streaming aggregations / windows — per-minute counts, sliding-window rates, sessionization.
- + Stream joins and enrichment — join two topics, or a stream against a slowly-changing table.
- + Real-time analytics / alerting where batch latency is too slow.
- + Stateful complex-event processing (pattern detection) over a firehose.

THE DISTINCTIONS TO SAY

- › **Kafka vs Flink**: Kafka is transport + storage (the log); Flink is compute over it. "Kafka moves and retains; Flink windows, aggregates, and joins." Kafka Streams is the lighter embedded library for simpler per-app transforms.
- › **Event-time vs processing-time**: event-time + watermarks is correct under out-of-order / late data; processing-time is simpler but wrong when events arrive late. Naming watermarks is the signal you actually understand streaming.
- › **Stream vs batch**: Flink treats batch as a bounded stream; reach for it when you need continuous low-latency results, not a nightly job.

AVOID / CAREFUL WHEN

- Simple stateless transform or fan-out — Kafka consumers / Kafka Streams are lighter.
- Small volume or an occasional job — a cron batch is far less operational weight.
- Large keyed state is real ops: state-backend sizing, checkpoint tuning, savepoint / restore on redeploy.

NUMBERS TO ANCHOR

Latency	ms → sub-second
Guarantee	exactly-once state
Time model	event-time + watermarks
State	local + checkpointed

CAP / CONSISTENCY

Gives exactly-once *state* internally via checkpoint-and-replay; end-to-end exactly-once needs a transactional / idempotent sink (two-phase-commit connector) or the output is at-least-once. Same caveat as Kafka EOS: the guarantee holds within the processing boundary; external side effects still need idempotency.

INTERVIEW LINE

When I need to compute over a stream — windowed aggregations, joins, sessionization in real time — I reach for Flink, not a Kafka consumer with a hand-rolled counter. It keeps large keyed state locally, checkpoints it for exactly-once recovery, and uses event-time watermarks so late events still land in the right window. End-to-end exactly-once still needs an idempotent sink.

PUSHBACK / WHEN IT FLIPS

Flink is real operational weight — state backends, checkpoints, savepoints. If the transform is stateless or the window is trivial, Kafka Streams or plain consumers are simpler. It earns its cost only when you have large, long-lived, keyed state and need correctness under out-of-order events — name the window and the state, or you don't need Flink.

07 Message Queue

SQS / RabbitMQ. Per-message work dispatch with ack, redelivery, and dead-lettering. The "distribute tasks to workers" primitive — distinct from Kafka's replayable log.

MECHANISM

A broker holds messages; workers pull (or are pushed) one, process it, and **ack** to delete it. No ack (worker crash/timeout) → the message becomes visible again and is redelivered. After N failed attempts it goes to a **dead-letter queue (DLQ)** for inspection. Unlike a log, a consumed+acked message is gone — there's no offset to rewind, no replay, no second consumer group reading the same message.

REACH FOR IT WHEN

- + Task/job dispatch to a worker pool — resize emails, thumbnails, webhooks, background jobs.
- + You want per-message retry + DLQ without building it (the broker owns redelivery).
- + Priority or delayed/scheduled delivery (native in RabbitMQ / SQS).
- + Decoupling where you don't need replay or fan-out — competing consumers draining one queue.

LOG VS. QUEUE — THE DISTINCTION TO SAY

- **Kafka (log):** retained, replayable, multiple independent consumer groups, ordering per partition. "Many readers, rewindable history."
- **Queue (SQS/RabbitMQ):** consumed-and-deleted, per-message ack/retry/DLQ, work distributed across competing consumers. "One logical reader, work drains."
- **Rule of thumb:** need replay or multiple consumers of the same event → log. Need work distribution with retry/DLQ → queue.
- ● **Caveat worth voicing:** Kafka share groups (KIP-932) now give per-message ack and redelivery, so that half of the distinction is softer than it was. Priority, delay, and mature DLQ tooling are still queue territory.

AVOID / CAREFUL WHEN

- You need to replay history or add a new consumer that sees past events — that's a log.
- Strict global ordering — most queues don't guarantee it (SQS standard is best-effort; SQS FIFO trades throughput for order).
- Very high fan-out to many independent subscribers — pub/sub or a log fits better.

NUMBERS TO ANCHOR

Delivery	≥ once (FIFO: ~exactly)
Ordering	best-effort (FIFO opt-in)
Retry	built-in (vis. timeout)
DLQ	built-in after N attempts
Replay	none (consumed = gone)

CAP / CONSISTENCY

Managed queues (SQS) prioritize availability and at-least-once delivery; ordering and dedup are opt-in (FIFO) at a throughput cost. Design consumers to be idempotent regardless — redelivery is normal, not exceptional.

INTERVIEW LINE

For distributing background work to a pool with automatic retry and dead-lettering, I use a task queue, not Kafka. Workers ack on success; a crash just redelivers after the visibility timeout, and poison messages fall to a DLQ. I reach for Kafka instead only when I need replay or multiple independent consumers of the same stream.

PUSHBACK / WHEN IT FLIPS

The mistake is defaulting to Kafka for everything "async." If there's one logical consumer draining work and you want retry/DLQ semantics for free, a queue is simpler and cheaper to operate. Flip to a log the moment you need replay, event fan-out, or an audit trail of what happened.

08 S3 / Blob Storage

Effectively infinite, cheap, durable bytes over HTTP. The rule: large binaries live here, your DB stores the pointer, and the bytes never transit your API tier.

MECHANISM

Key→object store over HTTP, ~11-nines durability. Not a filesystem — flat keyspace, objects are immutable blobs (overwrite, don't edit). Standard classes replicate across ≥ 3 AZs (One Zone classes deliberately don't — cheaper, less durable). Tiered storage (hot → infrequent → archive) trades retrieval latency for cost. **Presigned URLs** let clients read/write directly with time-limited credentials — your server signs, the client transfers. **Multipart upload** splits large objects into parts (parallel, resumable).

REACH FOR IT WHEN

- + Any large binary: video, images, uploads, backups, ML artifacts, logs.
- + Static assets to serve via CDN (S3 as origin).
- + Data-lake / event archive (cheap, queryable later via Athena/etc).
- + Decoupling upload transfer from your app servers entirely.

THE TWO PATTERNS TO SAY

- › **Presigned direct upload** — client PUTs straight to S3; API only issues URLs. Bytes bypass your tier.
- › **Multipart for big files** — one presigned URL per part, parallel, per-part retry, resumable via `ListParts`. Client picks part size subject to 5 MB min / 10k parts max.
- › ● **Conditional writes** — `If-None-Match: *` writes only if the key is absent (Aug 2024); `If-Match: <etag>` is compare-and-swap against the current version (Nov 2024); conditional copy followed in Oct 2025. Failure is a 412. This is what makes S3-only leader election, optimistic locking, and lock-free table formats possible with no separate coordination service.

NUMBERS TO ANCHOR

Durability	~11 nines (multi-AZ)
Single PUT cap (S3)	5 GB
Max object size	5 TB (multipart)
Multipart part min / max	5 MB / 10k parts
First-byte latency	tens of ms

CAP / CONSISTENCY

Strongly read-after-write consistent for all operations since Dec 2020 — a GET after a PUT returns the latest object. What it does *not* have: cross-object transactions, multi-key atomicity, or querying inside objects. ● It *does* now have single-object conditional writes, which is genuine compare-and-swap on one key. It's a durable key→blob store with a CAS primitive, not a database.

INTERVIEW LINE

Large binaries go to blob storage and the DB keeps only the key. Clients upload via presigned URLs so gigabytes never touch my API tier, and anything big uses multipart — one URL per part, parallel, resumable. I front it with a CDN for reads and drive state transitions off object-created events rather than trusting the client's "done."

PUSHBACK / WHEN IT FLIPS

S3 is now strongly consistent *and* supports conditional writes, so two old talking points are dead: the eventual-consistency caveat and "you need a lock service to coordinate writers on S3." The real limits are no multi-object atomicity and no querying inside objects. Don't store data you'll filter/join on as blobs. And it's the pointer-in-DB pattern: an object with no DB row is an orphan you pay for — reconcile with a sweeper.

09 CDN

Cache bytes at edge PoPs close to users. The answer to "global users" and "read-heavy static/cacheable content." Turns 20k QPS for one object into ~one origin fetch per PoP.

MECHANISM

Geographically distributed edge caches. User hits the nearest PoP; on a cache hit it serves locally (low latency, zero origin load). On a miss it fetches from origin, caches per TTL/headers, serves. Cache-Control / ETag govern freshness. Collapses read fan-out: N users pulling the same object = ~1 origin fetch per PoP per TTL. Also absorbs traffic spikes and DDoS at the edge.

REACH FOR IT WHEN

- + Static assets: JS/CSS/images, video segments, downloads.
- + Global read audience for the same content.
- + A hot cacheable value read enormously — e.g. the waiting-room "admitted" counter cached 2s collapses 500k polls to near-zero origin.
- + Shielding origin from spikes.

AVOID / CAREFUL WHEN

- Personalized / per-user responses — low hit rate, little benefit (edge compute is the escape hatch).
- Highly dynamic data where staleness is unacceptable.
- **Invalidation is the hard part** — short TTL (let it expire) beats active purge where you can tolerate seconds of staleness.

NUMBERS TO ANCHOR

Edge hit latency	single-digit → tens ms
Origin offload	often > 95%
Invalidation	TTL > active purge

CAP / CONSISTENCY

Deliberately eventually consistent — the edge serves possibly-stale content bounded by TTL. That staleness is the tradeoff you accept for latency and origin offload; design around "what's cacheable and for how long," not "how do I make the edge fresh."

INTERVIEW LINE

A CDN pushes cacheable bytes to edge PoPs so global reads hit a nearby cache instead of my origin — that's how one popular object survives 20k QPS at ~one origin fetch per PoP. I lean on short TTLs over active invalidation whenever seconds of staleness are acceptable, which is most static and near-static content.

PUSHBACK / WHEN IT FLIPS

A CDN does nothing for personalized or write paths — don't wave it at a dynamic problem. The real design question is always invalidation: reason about what's cacheable and for how long. Staleness tolerance is the actual variable.

10 Vector Database

Approximate nearest-neighbor search over embedding vectors. The retrieval half of RAG: "find the k chunks most semantically similar to this query."

MECHANISM

ANN index over high-dim vectors. Text/images → embeddings (e.g. 768–3072 dims) via a model. Query is embedded the same way; the DB finds nearest neighbors by cosine/dot distance. Exact search is $O(N)$ per query, so it uses an approximate index — usually **HNSW** (navigable small-world graph): logarithmic-ish search, tunable recall vs. latency. Metadata filtering (tenant, date, ACL) runs alongside vector search — critical and easy to get wrong.

REACH FOR IT WHEN

- + RAG: ground an LLM in your corpus — retrieve top-k chunks, stuff into context.
- + Semantic search (meaning, not keywords).
- + Recommendations / dedup / similarity by embedding.

PIPELINE TO SAY

- **Ingest:** chunk → embed → upsert with metadata.
- **Query:** embed → ANN top-k → filter by ACL → rerank → into prompt.
- Chunking strategy and reranking move quality more than the DB choice.

AVOID / CAREFUL WHEN

- Keyword/exact-match is what users want — BM25/Elasticsearch may beat or hybridize with vectors.
- Small corpus — a library (FAISS) or pgvector in Postgres beats a new system.
- **Permission leakage:** embeddings ignore ACLs. Filter by permission at query time or you retrieve across tenants.

NUMBERS TO ANCHOR

Embedding dims	~768–3072
Raw vector size	dims × 4 B (float32)
Quantization	int8 ~4×, binary ~32×
1M × 1536-dim	~6 GB raw + graph
Index	HNSW (recall↔latency)

CAP / CONSISTENCY

Usually eventually consistent and read-optimized; a just-upserted vector may not be immediately searchable. Fine for RAG (corpus changes slowly). **The deployed memory footprint is the real gotcha** — the HNSW graph frequently exceeds the raw vectors, and quantization is how you fit large corpora in RAM.

INTERVIEW LINE

For RAG I embed and index chunks with HNSW, then at query time embed the question, pull top-k by cosine, filter by the user's ACL, and rerank before building the prompt. Chunking and reranking drive quality more than which vector DB. Below a few million vectors I'd just use pgvector; above that I'd budget for the index memory and consider quantization.

PUSHBACK / WHEN IT FLIPS

Vector search isn't always the answer — hybrid (BM25 + vector) often wins because pure semantic misses exact terms (names, IDs, codes). And the sharp failure mode is permissions: sensitive data in embeddings/logs and cross-tenant retrieval. Raise ACL-at-query-time unprompted — it's the People-Innovation kill-shot.

11 Workflow Orchestrator

Temporal / Step Functions. Durable multi-step workflows with fan-out/fan-in and exactly-once continuation. The answer to "coordinate N async tasks and know when all are done."

MECHANISM

Event-sourced state machine + replay. Every state transition is appended to a durable history log — that log is truth, not in-memory state. On crash, the workflow re-executes from the top, but completed steps return their *recorded* results from the log instead of re-running; real execution resumes only past the end of history. Fan-in is a read over the ordered log ("is there a completion for every scheduled task"), decided by a single evaluator — so no concurrent-counter race. Requires deterministic workflow code (no clocks/random outside activities).

REACH FOR IT WHEN

- + Fan-out/fan-in: spawn N tasks, run one step when all finish (transcode ladder → mark READY).
- + Long-running, multi-step, must survive crashes (order saga, provisioning).
- + Steps need retries, timeouts, compensation without hand-rolled state.
- + You'd otherwise build a status table + counter + reaper by hand.

AVOID / CAREFUL WHEN

- Single-step / fire-and-forget — a queue is enough.
- Ultra-low-latency synchronous paths (replay + persistence add overhead).
- Activities are at-least-once — the *continuation* is exactly-once, but activity code must be idempotent.

TWO FLAVORS + NUMBERS

Temporal	your code + replay
Step Functions	declarative JSON FSM
Fan-in	built-in primitive
Guarantee	exactly-once continuation

CAP / CONSISTENCY

The orchestrator's own durability comes from its backing store (Temporal: Cassandra/Postgres/MySQL; Step Functions: managed). What it gives you is **exactly-once continuation** — the workflow advances as if each step ran once — layered on at-least-once activity execution. The two are different; your activities still need idempotency.

INTERVIEW LINE

When I need to fan out N tasks and fire one step when all complete, I reach for an orchestrator instead of a hand-rolled counter-and-reaper. It persists every transition to a durable log and decides completion with a single reader over that ordered log, so the fan-in race is gone by construction. Continuation is exactly-once; I still make activities idempotent.

PUSHBACK / WHEN IT FLIPS

Don't reach for it for a single async job — that's a queue, and the replay/determinism model is real cognitive + operational cost. The mechanism to name even if you hand-roll: atomic DECR for "am I last," a guarded status transition for idempotent completion, and a reaper for stalls.

12 ZooKeeper / etcd

Strongly-consistent coordination for small metadata: leader election, distributed locks, config, membership. Not a data store — a consensus-backed source of truth for "who's the leader" and "who's alive." (etcd is the modern, Raft-based equivalent.)

MECHANISM

A replicated in-memory tree of znodes (etcd: a flat key space), kept consistent across a small ensemble by an atomic-broadcast/consensus protocol — ZAB for ZooKeeper, Raft for etcd. Writes go through a leader and commit only on a quorum (majority), so an ensemble of $2f+1$ tolerates f failures (5 nodes → survive 2). Three primitives carry all the use cases: **ephemeral znodes** vanish when a client's session heartbeat lapses (→ liveness/membership); **watches** notify clients of changes (→ config push, leader-change notification); **sequential znodes** give globally ordered names (→ fair locks and leader election).

REACH FOR IT WHEN

- + Leader election / single-writer coordination — who owns this shard, who runs the singleton cron.
- + Distributed locks / barriers that must be correct (ephemeral + sequential znodes).
- + Service discovery / membership — ephemeral nodes as a live registry.
- + Config that must be consistent and pushed to watchers on change.

THE DISTINCTIONS TO SAY

- > **Not a database:** kilobytes per znode, whole tree in memory. Store pointers and coordination state, never bulk data.
- > **Consistency over availability:** writes need quorum, so a minority partition can't write — CP by design. That's the point; you want the lock wrong-proof, not always-available.
- > **Redis lock ≠ this:** SET NX + fencing is a best-effort lease (safe with a fencing token, but availability-first); ZooKeeper/etcd give consensus-backed correctness. Name the difference when a double-holder is a correctness bug.
- > **Increasingly invisible — and increasingly etcd:** ZooKeeper was the thing under other systems for a decade, but that's receding. Kafka 4.0 (2025) removed it in favour of its own Raft implementation (KRaft); HBase still uses it; and for most engineers today consensus coordination is met as etcd under Kubernetes.

NUMBERS TO ANCHOR

Ensemble	3 or 5 ($2f+1$)
Data size	KB per znode
Writes	quorum consensus
Consistency	linearizable writes (CP)

CAP / CONSISTENCY

Strongly consistent / CP: linearizable writes via quorum; reads are sequentially consistent and can lag slightly (use a sync / quorum read for linearizable reads). It deliberately sacrifices availability in the minority partition — the ensemble stops accepting writes rather than risk split-brain, which is exactly what you want backing a lock or leader election.

INTERVIEW LINE

For leader election, locks, or membership I want consensus-backed coordination — ZooKeeper or etcd — not a Redis lock. Ephemeral znodes give me liveness, sequential znodes give ordered fair locks, and quorum writes make it CP so the lock can't split-brain. I keep only kilobytes of coordination metadata there; the real data lives elsewhere.

PUSHBACK / WHEN IT FLIPS

Reaching for ZooKeeper directly is now rare, and the trend is one-directional: Kafka, its most famous dependent, dropped it entirely in 4.0. etcd (Raft, behind Kubernetes) is the modern default — propose that, and treat ZooKeeper as the name for the pattern rather than the deployment you'd choose today. For a best-effort lock where an occasional double-run is tolerable, a Redis lease is simpler and cheaper; escalate to consensus only when a double-leader is a correctness bug, not a nuisance. And never put throughput or bulk data on it — every write is a consensus round.

13 Elasticsearch

An inverted index for full-text and faceted search. A secondary read model you feed from your primary store — not your source of truth.

MECHANISM

Inverted index (Lucene) + distributed shards. Text is analyzed (tokenized, lowercased, stemmed) into terms; the inverted index maps term → documents, so full-text queries are fast. Relevance ranked by **BM25**. Indexes shard + replicate for scale and HA. Near-real-time, not immediate — a refresh interval (default ~1s) gates visibility. Async-fed from your primary DB via CDC or dual-write.

REACH FOR IT WHEN

- + Full-text search with relevance ranking (search bars, docs, logs).
- + Faceted / filtered search — aggregations across many fields (e-commerce filters).
- + Log / observability search at scale (the ELK stack).
- + Hybrid with vectors for semantic + keyword.

AVOID / CAREFUL WHEN

- As a primary store — it's a derived index; rebuild from truth. Don't put data only here.
- You need strong consistency / transactions — near-real-time and no ACID.
- Simple exact lookups — a DB index is cheaper and simpler.
- Keeping it in sync is the real cost — CDC pipeline + reconciliation.

NUMBERS TO ANCHOR

Visibility	near-real-time (~1s)
Ranking	BM25
Role	derived read model

CAP / CONSISTENCY

AP-leaning and eventually consistent with your primary by construction (async-fed, ~1s refresh). Treat divergence as normal: design the sync pipeline, a reconciliation path, and the ability to rebuild the index from the source of truth at any time.

INTERVIEW LINE

Full-text and faceted search go to Elasticsearch as a secondary read model fed from my primary store via CDC — the inverted index and BM25 give relevance ranking a B-tree can't. It's near-real-time and not my source of truth, so I design the sync pipeline and a reconciliation path, and I can rebuild the index from the primary at any time.

PUSHBACK / WHEN IT FLIPS

The trap is treating it as a database. It's a derived index — the hard part is the sync pipeline and its consistency, not the query. For semantic search, hybrid BM25 + vector usually beats either alone. For plain exact-match, a Postgres index is simpler and consistent.

14 OLAP / Columnar Store

ClickHouse / Snowflake / BigQuery. Scan-and-aggregate over billions of rows. The read model behind dashboards, funnels, and ad-hoc analytics — never your transactional store.

MECHANISM

Column-oriented storage: each column is stored contiguously and compressed, so a query touching 3 of 200 columns reads roughly 3/200 of the bytes — the whole reason a `GROUP BY` over a billion rows is feasible. Adjacent values in a column are similar, so compression is high. **Vectorized execution** processes batches of column values per instruction instead of row-at-a-time. Data is sorted/partitioned (usually by time) with per-block min/max, so range predicates prune entire blocks without reading them. Fed asynchronously from the primary store via CDC or batch ETL.

REACH FOR IT WHEN

- + Dashboards and reporting over large event/fact tables — aggregations across billions of rows.
- + Ad-hoc analytical slicing where the query shape isn't known in advance.
- + Funnel, retention, cohort analysis; time-series rollups.
- + You're about to point heavy analytics at a Postgres replica and melt it.

THE DISTINCTIONS TO SAY

- › **OLTP vs OLAP:** a row store is optimized for "all columns of one row"; a column store for "one column of all rows." That single sentence is the whole tradeoff.
- › **vs Flink:** Flink pre-computes fixed aggregations continuously as events arrive; OLAP stores raw rows and answers *unanticipated* questions at read time. Real systems use both.
- › **vs Elasticsearch:** ES is an inverted index for text relevance; OLAP is scan + aggregate over typed columns.
- › **ClickHouse vs warehouse:** ClickHouse for sub-second, real-time-ingest, user-facing analytics; Snowflake/BigQuery for elastic, separated storage/compute, higher-latency internal analysis.

AVOID / CAREFUL WHEN

- Point lookups or per-row updates — updates mean rewriting/merging parts; use a row store or KV.
- Transactions, foreign keys, referential integrity — not the model.
- Results must be immediately consistent with the primary — it's async-fed and insert-batched.
- Small data — a Postgres index or materialized view is simpler and one fewer pipeline.

NUMBERS TO ANCHOR

Compression	~5-10× typical
Scan rate	100M+ rows/s/node
Query latency	sub-second → seconds
Insert pattern	batched, not row-at-a-time
Role	derived read model

CAP / CONSISTENCY

Eventually consistent with the primary by construction, and typically without multi-statement transactions. Same discipline as Elasticsearch: it's derived, so design the ingest pipeline, tolerate seconds-to-minutes of lag, and keep the ability to rebuild from the source of truth.

INTERVIEW LINE

Analytics goes to a columnar store fed by CDC, not to a read replica — column storage plus vectorized execution means an aggregation over billions of rows only reads the columns it touches, and time partitioning prunes most blocks before they're read. I batch inserts rather than writing row-at-a-time, accept seconds of lag behind the OLTP store, and keep Postgres as truth.

PUSHBACK / WHEN IT FLIPS

Don't add it before the data is big — Postgres with a materialized view or a rollup table absorbs a surprising amount, and a separate analytics store is another pipeline to keep in sync and reconcile. The real trigger is query shape plus volume: unbounded scans and aggregations over a fact table your OLTP primary can't serve without competing with production traffic.

15 Decision matrix

Collapse the fourteen stores into one glance. Match the workload on the left to the default pick; the "because" is the one-line justification.

WORKLOAD	DEFAULT PICK	BECAUSE
Transactions, joins, ad-hoc queries	PostgreSQL	ACID + query flexibility; the default until something breaks
Keyed lookups, global reads, huge scale (self-run)	Cassandra	auto-rebalance + multi-region writes; you don't need joins
Keyed lookups at scale, serverless / zero-ops	DynamoDB	Dynamo-lineage auto-shard, managed, per-request cost; design the partition key
Write firehose / > ~50k writes/s to one table	Cassandra / DynamoDB / TSDB	LSM absorbs append-heavy volume one Postgres primary can't
Cache / counter / lock / TTL'd ephemeral state	Redis	in-memory, atomic ops, microsecond latency
Decouple + replay + multiple consumers	Kafka	retained ordered log, rewindable, fan-out
Windowed aggregation / stream join / sessionize	Flink	stateful stream compute, event-time watermarks, exactly-once state
Background jobs with retry + DLQ	Queue (SQS / RabbitMQ)	per-message ack/redelivery; no replay needed
Large binaries (video, images, uploads)	S3 / Blob	infinite cheap durable bytes; keep the pointer in the DB
Global static / hot read-heavy content	CDN	serve from the edge; collapse read fan-out
Semantic search / RAG retrieval	Vector DB	ANN over embeddings; hybrid with BM25 for exact terms
Coordinate N tasks, fan-in, sagas	Orchestrator	durable fan-out/fan-in, exactly-once continuation
Leader election / lock / membership / consistent config	ZooKeeper / etcd	consensus-backed coordination (CP); not a data store
Full-text / faceted search	Elasticsearch	inverted index + BM25; a derived read model, not truth
Dashboards / ad-hoc aggregation over billions of rows	OLAP / columnar	column scans + vectorized execution; derived from OLTP via CDC

THE META-RULE

Most systems are **Postgres + one or two specialists**: Postgres as the transactional source of truth, plus a cache (Redis), a blob store (S3), a CDN, and maybe a search, vector, or analytics index fed from Postgres via CDC. Reach for Cassandra / DynamoDB / Kafka / Flink / an orchestrator only when a *named requirement* — a specific hot table past ~15–50k writes/s, global write scale, replay/fan-out, real-time stream compute, durable multi-step coordination — forces it. Naming the one store as truth and treating the rest as derived is the coherence signal.



Cross-cutting principles

The five rules that reappear on every page above. If you only carry one thing into the room, carry these.

01 Idempotency is non-negotiable

anywhere delivery is at-least-once (Kafka, queues, orchestrator activities, Flink sinks, retried writes).

02 One source of truth

; everything else (cache, search index, vector store, analytics store, denormalized copy) is derived and must be rebuildable + reconciled.

03 Reads tolerate staleness; writes must be correct

— the asymmetry behind CDNs, replicas, caches, and search indexes.

04 Move a guarantee out of the database only when arrival rate makes the DB physically unable to provide it

— the same rule for sharding (~15–50k writes/s to one table), Redis holds, and counter offloading (~500–1k/s to one hot row).

05 Consensus (ZooKeeper/etcd) for correctness-critical coordination; a lease (Redis) when an occasional double is tolerable

— pick the strength of guarantee the failure demands.